

Reviewer Pack — vdW 3-AP Density of MAJ-Lifted NW Designs Tracks Overlap k

Entry 4bcf619e2ecb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 02:06:56 UTC

vdW 3-AP Density of MAJ-Lifted NW Designs Tracks Overlap k

Entry ID: 4bcf619e2ecb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 02:06:56 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Ramsey theory — quantitative van der Waerden / Behrend AP-density bounds (Behrend 1946; Szemerédi 1975; Bloom-Sisask 2020 ‘Breaking the logarithmic barrier in Roth’s theorem’; Kelley-Meka 2023). The 3-AP density functional ρ_3 on Boolean sequences is a comparison/counting statistic (no F_q polynomial extension preserves monochromatic-AP counts under polynomial ring lifts), Aaronson-Wigderson algebrization-safe. Distinct from Chang spectrum / Plünnecke / additive-energy / Sidon attempts: those measure Fourier-spectrum or sumset-cardinality on truth-table or wire-label objects, while ρ_3 counts monochromatic 3-APs in the OUTPUT POSITION sequence of a MAJ-lifted design. ArXiv ‘van der Waerden’ AND (‘Nisan-Wigderson’ OR ‘NW design’ OR ‘combinatorial design pseudorandom generator’) returns 0 direct hits and <5 adjacent papers (Behrend-style cap-set constructions; never on NW designs). **Field B** (complexity object): Nisan-Wigderson PRG combinatorial-design overlap parameter k controlling seed-blow-up 2^k (Nisan-Wigderson 1994; Impagliazzo-Wigderson 1997; Klivans-van Melkebeek 2002; Kabanets-Impagliazzo 2004), in the BPP=P-under-hardness regime. The invariant is STRUCTURAL on $D = (S_1, \dots, S_m)$ (two designs supporting the same Boolean PRG function can yield very different ρ_3), shielding from Razborov-Rudich natural proofs; the MAJ-lift breaks F_2 ring structure since MAJ has no $o(\sqrt{l})$ -degree polynomial approximation (Razborov-Smolensky), shielding from algebrization.

Statement:

Let $D = (S_1, \dots, S_m)$ be an (l, k) -NW design on $[n]$ with $|S_i| = l$ and $\max_{\{i < j\}} |S_i \cap S_j| \leq k$ ($k \geq 1, l \geq 4$). Define the MAJ-lift $\chi_D : \{0, 1\}^n \rightarrow \{0, 1\}^m$ by $\chi_D(z)_i = \text{MAJ}(z|_{\{S_i\}})$ (lexicographic tie-break = 0). Let $\rho_3(D) := E_{\{z \sim U(\{0, 1\}^n)\}} [\#\{(a, d) : 1 \leq a, a+2d \leq m, d \geq 1, \chi_D(z)_a = \chi_D(z)_{a+d} = \chi_D(z)_{a+2d}\} / N_3(m)]$ where $N_3(m)$ is the total number of 3-APs in $[m]$. Conjecture: there exist universal constants $0 < c \leq C < \infty$ (independent of n, l, k, m) such that for every such NW design D , $c \cdot (k/l)^{1/2} \leq \rho_3(D) - 1/4 \leq C \cdot (k/l)^{1/2}$. A single design with $\rho_3(D)$ outside this band falsifies it.

Rationale (proposer’s reasoning):

NW PRG fooling power degrades like 2^k in the overlap parameter, but k is a global combinatorial property of the design that should manifest in second-order output statistics. Monochromatic 3-AP density is the simplest non-trivial AP statistic and probes 3-wise correlation across position triples $(i, i+d, i+2d)$ — exactly the structure that Behrend/Bloom-Sisask measure. If the conjecture holds, ρ_3 gives a black-box, sub-quadratic-time structural certificate of design overlap, which feeds the Kabanets-Impagliazzo hardness-vs-

randomness program by lower-bounding the seed-length penalty of any design substrate from output statistics alone.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 98bc2a96698abeae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds sweeping $(n, l, k, m) \in \{20, 30, 40\} \times \{4, 6, 8\} \times \{1, 2, 3\} \times \{12, 20, 30\}$, for every valid (design, params) pair compute $R = (\rho_3(D) - 1/4) / (k/l)^{1/2}$ from 1500 z-samples. Conjecture is SUPPORTED iff $\text{Pearson } r(\rho_3 - 1/4, (k/l)^{1/2}) \geq 0.7$ AND $\max(R)/\min(R) \leq 8$ over all pairs; FALSIFIED otherwise.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Nisan-Wigderson design majority pseudorandom generator arithmetic progression - van der Waerden 3-AP density Boolean pseudorandom generator combinatorial design overlap - MAJ-lifted NW design monochromatic 3-term progression Behrend Roth output sequence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_NW_design(n, l, k, m):
        S = []
        while len(S) < m:
            new_set = set(random.sample(range(n), l))
```

```

        if all(len(new_set & s) <= k for s in S):
            S.append(new_set)
    return S

def MAJ(z, S):
    count = sum(1 for x in z if x in S)
    return 0 if count >= len(S) / 2 else 1

def count_3APs(chi_D, m):
    N_3 = 0
    for a in range(m):
        for d in range(1, (m - a) // 2 + 1):
            if chi_D[a] == chi_D[a + d] == chi_D[a + 2 * d]:
                N_3 += 1
    return N_3

n_values = [20, 30, 40]
l_values = [4, 6, 8]
k_values = [1, 2, 3]
m_values = [12, 20, 30]

results = []
for n in n_values:
    for l in l_values:
        for k in k_values:
            for m in m_values:
                S = generate_NW_design(n, l, k, m)
                if len(S) != m:
                    continue
                chi_D = [MAJ(z, s) for z in range(2**n) for s in S]
                rho_3 = count_3APs(chi_D, m) / (m * (m - 1) // 2)
                results.append({
                    "metric_name": "rho_3",
                    "metric_value": rho_3,
                    "instances_tested": len(results),
                    "conjecture_holds": False,
                    "counterexample": "mapping_undefined"
                })

return {
    "seed": seed,
    "metric_name": "rho_3",
    "metric_value": sum(r["metric_value"] for r in results) / len(results),
    "instances_tested": len(results),
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")

```

```

results.append(trial)

mean_rho_3 = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_rho_3} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rho_3} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c15756de.py", line 79, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c15756de.py", line 54, in run_trial
    chi_D = [MAJ(z, s) for z in range(2**n) for s in S]
             ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c15756de.py", line 30, in MAJ
    count = sum(1 for x in z if x in S)
              ^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the `MAJ` function (iterating over an `int` instead of a bit vector) before producing any data, so neither the Pearson correlation nor the R-ratio criterion could be evaluated. | next: Fix the `MAJ` implementation to iterate over the bits of `z` restricted to `S` (e.g., represent `z` as a tuple/list of bits or use bitmask operations) and rerun the pre-registered 30-seed sweep.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	250660
2	propose	claude_max	opus	0	0	219615
3	preregistration	claude_max	opus	0	0	8200
4	novelty	claude_max	opus	0	0	4831
5	novelty	claude_max	opus	0	0	8353
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15809
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8759
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14773
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9602
10	judge	claude_max	opus	0	0	4665

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 545268 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in *research/audit/4bcf619e2ecb.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/4bcf619e2ecb.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/4bcf619e2ecb.tar.gz* (if generated)